

HW1 Object Detection Report

NYCU_314706007 莊竣堯

1. Model Description

1.1 Overview

Tried different models from papers on dense object detection task, lastly choose to use **YOLOv8m** structure for best result. I run the model on Google Colab, and YOLOv8m is the biggest YOLOv8 model that can put inside 15GB vram, and also has efficient speed and quality.

1.2 Architecture

- YOLOv8m follows **Backbone–Neck–Head** structure.
 - **Backbone:** feature extraction (Conv, C2f, SPPF)
 - **Neck:** multi-scale feature fusion (upsampling + concatenation)
 - **Head:** detection at three scales (P3, P4, P5)
- **Custom modifications**
 - Changed input size: 640
 - Modified number of classes to one class

	from	n	params	module	arguments
0	-1	1	1392	ultralytics.nn.modules.conv.Conv	[3, 48, 3, 2]
1	-1	1	41664	ultralytics.nn.modules.conv.Conv	[48, 96, 3, 2]
2	-1	2	111360	ultralytics.nn.modules.block.C2f	[96, 96, 2, True]
3	-1	1	166272	ultralytics.nn.modules.conv.Conv	[96, 192, 3, 2]
4	-1	4	813312	ultralytics.nn.modules.block.C2f	[192, 192, 4, True]
5	-1	1	664320	ultralytics.nn.modules.conv.Conv	[192, 384, 3, 2]
6	-1	4	3248640	ultralytics.nn.modules.block.C2f	[384, 384, 4, True]
7	-1	1	1991808	ultralytics.nn.modules.conv.Conv	[384, 576, 3, 2]
8	-1	2	3985920	ultralytics.nn.modules.block.C2f	[576, 576, 2, True]
9	-1	1	831168	ultralytics.nn.modules.block.SPPF	[576, 576, 5]
10	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
11	[-1, 6]	1	0	ultralytics.nn.modules.conv.Concat	[1]
12	-1	2	1993728	ultralytics.nn.modules.block.C2f	[960, 384, 2]
13	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
14	[-1, 4]	1	0	ultralytics.nn.modules.conv.Concat	[1]
15	-1	2	517632	ultralytics.nn.modules.block.C2f	[576, 192, 2]
16	-1	1	332160	ultralytics.nn.modules.conv.Conv	[192, 192, 3, 2]
17	[-1, 12]	1	0	ultralytics.nn.modules.conv.Concat	[1]
18	-1	2	1846272	ultralytics.nn.modules.block.C2f	[576, 384, 2]
19	-1	1	1327872	ultralytics.nn.modules.conv.Conv	[384, 384, 3, 2]
20	[-1, 9]	1	0	ultralytics.nn.modules.conv.Concat	[1]
21	-1	2	4207104	ultralytics.nn.modules.block.C2f	[960, 576, 2]
22	[15, 18, 21]	1	3776275	ultralytics.nn.modules.head.Detect	[1, [192, 384, 576]]

YOLOv8m summary: 169 layers, 25,856,899 parameters, 25,856,883 gradients, 79.1 GFLOPs

2. Implementation Details

2.1 Dataset

Skipping 4 annotation entries without images: 00000604, 00000605, 00000606, 00000607

train images: 1140 | val images: 126 | total boxes: 38619, train/val ratio: 0.1

test images: 1864

2.2 Preprocessing & Augmentation

- Preprocessing:

Resize / Letterbox	Resizes the image so that its longest side = 640 while keeping aspect ratio. Pads with gray (114,114,114) to make it square.
Convert to RGB	Converts BGR (OpenCV default) → RGB.
Normalize	Divides pixel values by 255 → scales from [0,255] → [0,1] .
Convert to Tensor	Converts $H \times W \times 3$ to $3 \times H \times W$, and ensures float32.
Batch + Device transfer	Adds batch dimension, moves to CUDA, and applies half precision (FP16) if enabled.

- Augmentation:

Mosaic (p=1.0)	Combines 4 images into one large image, random placements. Strongly improves small-object recall.
MixUp (p=0.15)	Linearly blends two images and labels — regularization for dense scenes.
Random affine transform	Random rotation ($\pm 10^\circ$), translation ($\pm 10\%$), scale ($0.5\text{--}1.5\times$), and shear ($\pm 2^\circ$).
Horizontal flip (p=0.5)	Mirror the image and swap bounding boxes horizontally.
HSV color-space augment	Random hue ($\pm 5^\circ$), saturation ($\pm 30\%$), and value ($\pm 30\%$).
Brightness/Contrast jitter	Varies lighting slightly per batch.

2.3 Hyperparameters

Parameter	Value	Description
Image Size	640	Input resolution
Batch Size	16	Number of images per batch
Epochs	50	Total training epochs
Optimizer	Adam / SGD	With learning rate schedule
Learning Rate	0.01	Initial LR
Momentum	0.937	As used in YOLOv8 default
Weight Decay	0.0005	Regularization term

2.4 Loss Functions

- **Bounding box loss:** CIOU + Distribution Focal Loss
- **Classification loss:** Binary Cross-Entropy (BCE)
- **Confidence loss:** BCE

2.5 Training Strategy

- In validation, set **confidence threshold to 0.01**, and **IOU overlap threshold to 0.6** to get more bounding boxes. This is because the training data has dense object, so I think the more boxes we have the more likely we predict correctly and achieve higher score in Kaggle competition. The result also shows some improvement compare to different threshold.

3. Result Analysis

3.1 Quantitative Results

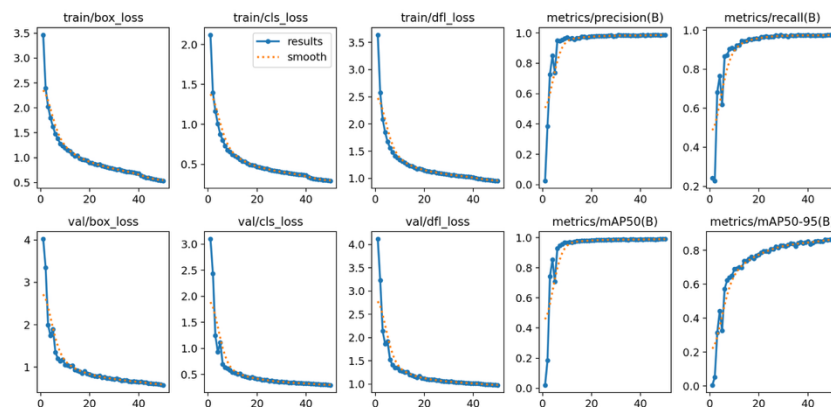
Model	mAP@50	mAP@50-95	Precision	Recall
YOLOv8n	0.985	0.832	0.983	0.970
YOLOv8m	0.990	0.867	0.986	0.975

Both YOLOv8 variants achieved excellent accuracy on the pig-detection dataset.

The medium model (YOLOv8m) performed slightly better than the nano version, showing higher mAP and balanced precision (0.986) and recall (0.975).

Its improvement mainly comes from the larger network capacity and effective augmentations such as Mosaic, MixUp, and HSV adjustments.

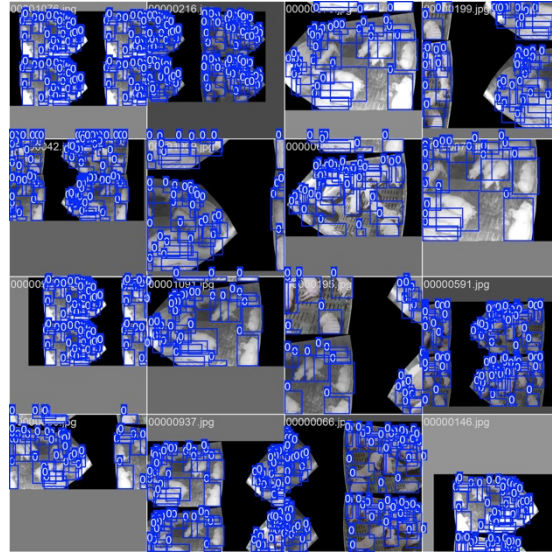
3.2 Qualitative Results



(a) Training curves – learning behavior

Loss curves decrease smoothly and stabilize after ~40 epochs, while precision, recall, and mAP steadily approach 1.0.

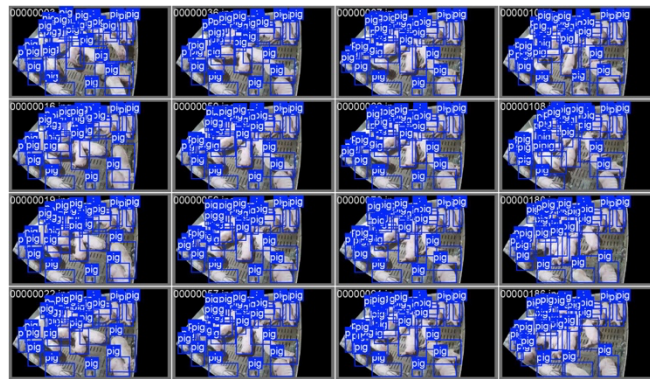
The close alignment between training and validation curves indicates good convergence and strong generalization.



(b) Training-phase detections

Mosaic-composed images show consistent, high-confidence detections across varied scales and lighting.

This confirms that data augmentation is functioning correctly and helps the model handle overlaps and occlusions effectively.



(c) Validation-phase detections

Validation images display accurate, single bounding boxes for each pig with confidence above 0.95.

The model generalizes well from augmented data to real scenes, achieving precise localization and minimal duplicate or missed detections.

Error analysis:

- Occasional misses occur for **piglets** or animals at image borders.
- Mild confidence drops appear under **motion blur** or **low illumination**.

These residual errors are consistent with the quantitative gap between $\text{mAP}@50$ (almost 1.0) and $\text{mAP}@50-95$ (≈ 0.86), reflecting tighter IoU thresholds being more sensitive to small-box localization.

4. Short Conclusion

This project demonstrated the successful customization of YOLOv8m for pig detection. The model was trained directly on a pig-specific dataset with tailored preprocessing, augmentations (Mosaic, MixUp, HSV, affine), and optimized hyperparameters for stable convergence. The final model achieved **$\text{mAP}@50 = 0.99$** , **$\text{mAP}@50-95 = 0.867$** , **precision = 0.986**, and **recall = 0.975**, showing excellent detection accuracy across different lighting, poses, and enclosure conditions. Remaining errors mainly involve small piglets, overlapping individuals, or motion blur, suggesting that future improvements could include targeted small-object augmentation and higher-resolution inputs. Overall, the fine-tuned YOLOv8m delivers high accuracy, real-time inference, and efficiency, making it a strong candidate for smart livestock monitoring and precision-farming applications.